

Lesson 3: Email Assistant with Semantic Memory

We previously built an email assistant that:

- Classifies incoming messages (respond, ignore, notify)
- Drafts responses
- Schedules meetings

Now, we'll add memory.

We'll give the assistant the ability to remember details from previous emails.

 **Access requirements.txt , notebooks and other files:** 1) click on the "File" option on the top menu of the notebook and then 2) click on "Open".

 **Download Notebooks:** 1) click on the "File" option on the top menu of the notebook and then 2) click on "Download as" and select "Notebook (.ipynb)".

 For more help, please see the "Appendix – Tips, Help, and Download" Lesson.

 **Different Run Results:** The output generated by AI chat models can vary with each execution due to their dynamic, probabilistic nature. Don't be surprised if your results differ from those shown in the video.

Load API tokens for our 3rd party APIs

```
In [ ]: import os
        from dotenv import load_dotenv
        _ = load_dotenv()
```

Repeat setup from previous lesson

```
In [2]: profile = {
        "name": "John",
        "full_name": "John Doe",
        "user_profile_background": "Senior software engineer leading a team of 5
    }
```

```
In [3]: prompt_instructions = {
        "triage_rules": {
            "ignore": "Marketing newsletters, spam emails, mass company announce
```

```

        "notify": "Team member out sick, build system notifications, project
        "respond": "Direct questions from team members, meeting requests, cr
    },
    "agent_instructions": "Use these tools when appropriate to help manage J
}

```

```

In [4]: email = {
    "from": "Alice Smith <alice.smith@company.com>",
    "to": "John Doe <john.doe@company.com>",
    "subject": "Quick question about API documentation",
    "body": """
Hi John,

I was reviewing the API documentation for the new authentication service and

Specifically, I'm looking at:
- /auth/refresh
- /auth/validate

Thanks!
Alice""",
}

```

```

In [5]: from pydantic import BaseModel, Field
from typing_extensions import TypedDict, Literal, Annotated
from langchain.chat_models import init_chat_model

```

```

In [6]: llm = init_chat_model("openai:gpt-4o-mini")

```

```

In [7]: class Router(BaseModel):
    """Analyze the unread email and route it according to its content."""

    reasoning: str = Field(
        description="Step-by-step reasoning behind the classification."
    )
    classification: Literal["ignore", "respond", "notify"] = Field(
        description="The classification of an email: 'ignore' for irrelevant
        'notify' for important information that doesn't need a response, "
        "'respond' for emails that need a reply",
    )

```

```

In [8]: llm_router = llm.with_structured_output(Router)

```

```

In [9]: from prompts import triage_system_prompt, triage_user_prompt

```

```

In [10]: from langchain_core.tools import tool

```

```

In [11]: @tool
def write_email(to: str, subject: str, content: str) -> str:
    """Write and send an email."""
    # Placeholder response - in real app would send email
    return f"Email sent to {to} with subject '{subject}'"

```

```
In [12]: @tool
def schedule_meeting(
    attendees: list[str],
    subject: str,
    duration_minutes: int,
    preferred_day: str
) -> str:
    """Schedule a calendar meeting."""
    # Placeholder response - in real app would check calendar and schedule
    return f"Meeting '{subject}' scheduled for {preferred_day} with {len(attendees)} attendees"
```

```
In [13]: @tool
def check_calendar_availability(day: str) -> str:
    """Check calendar availability for a given day."""
    # Placeholder response - in real app would check actual calendar
    return f"Available times on {day}: 9:00 AM, 2:00 PM, 4:00 PM"
```

Define tools for managing memory

```
In [14]: from langgraph.store.memory import InMemoryStore
```

```
In [15]: store = InMemoryStore(
    index={"embed": "openai:text-embedding-3-small"}
)
```

/usr/local/lib/python3.11/site-packages/langgraph/store/base/embed.py:95: LangChainBetaWarning: The function `init\_embeddings` is in beta. It is actively being worked on, so the API may change.

```
    return init_embeddings(embed)
```

```
In [16]: from langmem import create_manage_memory_tool, create_search_memory_tool
```

```
In [17]: manage_memory_tool = create_manage_memory_tool(
    namespace=(
        "email_assistant",
        "{langgraph_user_id}",
        "collection"
    )
)
search_memory_tool = create_search_memory_tool(
    namespace=(
        "email_assistant",
        "{langgraph_user_id}",
        "collection"
    )
)
```

```
In [18]: print(manage_memory_tool.name)
```

manage_memory

```
In [19]: print(manage_memory_tool.description)
```

Create, update, or delete persistent MEMORIES to persist across conversations.

Include the MEMORY ID when updating or deleting a MEMORY. Omit when creating a new MEMORY – it will be created for you.

Proactively call this tool when you:

1. Identify a new USER preference.
2. Receive an explicit USER request to remember something or otherwise alter your behavior.
3. Are working and want to record important context.
4. Identify that an existing MEMORY is incorrect or outdated.

```
In [20]: manage_memory_tool.args
```

```
Out[20]: {'content': {'anyOf': [{'type': 'string'}, {'type': 'null'}],
  'default': None,
  'title': 'Content'},
  'action': {'default': 'create',
  'enum': ['create', 'update', 'delete'],
  'title': 'Action',
  'type': 'string'},
  'id': {'anyOf': [{'format': 'uuid', 'type': 'string'}, {'type': 'null'}],
  'default': None,
  'title': 'Id'}}
```

```
In [21]: search_memory_tool.name
```

```
Out[21]: 'search_memory'
```

```
In [22]: search_memory_tool.description
```

```
Out[22]: 'Search your long-term memories for information relevant to your current context.'
```

```
In [23]: search_memory_tool.args
```

```
Out[23]: {'query': {'title': 'Query', 'type': 'string'},
  'limit': {'default': 10, 'title': 'Limit', 'type': 'integer'},
  'offset': {'default': 0, 'title': 'Offset', 'type': 'integer'},
  'filter': {'anyOf': [{'type': 'object'}, {'type': 'null'}],
  'default': None,
  'title': 'Filter'}}
```

```
In [24]: agent_system_prompt_memory = """
< Role >
You are {full_name}'s executive assistant. You are a top-notch executive ass
</ Role >

< Tools >
You have access to the following tools to help manage {name}'s communication

1. write_email(to, subject, content) – Send emails to specified recipients
2. schedule_meeting(attendees, subject, duration_minutes, preferred_day) – S
3. check_calendar_availability(day) – Check available time slots for a given
4. manage_memory – Store any relevant information about contacts, actions, c
5. search_memory – Search for any relevant information that may have been st
```

```

</ Tools >

< Instructions >
{instructions}
</ Instructions >
"""

```

```

In [25]: def create_prompt(state):
        return [
            {
                "role": "system",
                "content": agent_system_prompt_memory.format(
                    instructions=prompt_instructions["agent_instructions"],
                    **profile
                )
            }
        ] + state['messages']

```

```

In [26]: from langgraph.prebuilt import create_react_agent

```

```

In [27]: tools= [
        write_email,
        schedule_meeting,
        check_calendar_availability,
        manage_memory_tool,
        search_memory_tool
    ]
    response_agent = create_react_agent(
        "anthropic:claude-3-5-sonnet-latest",
        tools=tools,
        prompt=create_prompt,
        # Use this to ensure the store is passed to the agent
        store=store
    )

```

```

In [28]: config = {"configurable": {"langgraph_user_id": "lance"}}

```

```

In [29]: response = response_agent.invoke(
        {"messages": [{"role": "user", "content": "Jim is my friend"}]},
        config=config
    )

```

```

In [30]: for m in response["messages"]:
        m.pretty_print()

```

===== Human Message =====

=====

Jim is my friend

===== Ai Message =====

=====

```
[{'text': "I'll help store this information about Jim in memory for future reference.", 'type': 'text'}, {'id': 'toolu_01ECTQonEqSU4h5GiE7F5C4o', 'input': {'content': "Jim is John Doe's friend."}, 'name': 'manage_memory', 'type': 'tool_use'}]
```

Tool Calls:

manage_memory (toolu_01ECTQonEqSU4h5GiE7F5C4o)

Call ID: toolu_01ECTQonEqSU4h5GiE7F5C4o

Args:

content: Jim is John Doe's friend.

===== Tool Message =====

=====

Name: manage_memory

created memory 2e51927d-2798-4afb-9551-d28f6a618c27

===== Ai Message =====

=====

I've recorded that Jim is your friend in my memory. This will help me better assist you with any future interactions or communications involving Jim. Is there anything specific you'd like me to help you with regarding Jim?

```
In [37]: response = response_agent.invoke(  
        {"messages": [{"role": "user", "content": "who is jim?"}]},  
        config=config  
    )
```

```
In [38]: for m in response["messages"]:  
        m.pretty_print()
```

```

===== Human Message =====
=====

who is jim?

===== Ai Message =====
=====

[{'text': 'Let me search through the stored memories to find any information
about Jim.', 'type': 'text'}, {'id': 'toolu_011sBHjRoVn6y7sHhc1EBCpP', 'input': {'query': 'Jim'}, 'name': 'search_memory', 'type': 'tool_use'}]
Tool Calls:
  search_memory (toolu_011sBHjRoVn6y7sHhc1EBCpP)
  Call ID: toolu_011sBHjRoVn6y7sHhc1EBCpP
  Args:
    query: Jim

===== Tool Message =====
=====

Name: search_memory

[{"namespace": ["email_assistant", "lance", "collection"], "key": "2e51927d-2798-4afb-9551-d28f6a618c27", "value": {"content": "Jim is John Doe's friend."}, "created_at": "2025-04-13T14:00:36.853164+00:00", "updated_at": "2025-04-13T14:00:36.853170+00:00", "score": 0.3853109317618744}]

===== Ai Message =====
=====

Based on the stored information, Jim is John Doe's friend. However, there isn't much detailed information available about him in the memories. If you'd like to know more specific details about Jim or would like me to remember additional information about him, please let me know and I can store that in the memory system.

```

```
In [39]: store.list_namespaces()
```

```
Out[39]: [('email_assistant', 'lance', 'collection')]
```

```
In [40]: store.search(('email_assistant', 'lance', 'collection'))
```

```
Out[40]: [Item(namespace=['email_assistant', 'lance', 'collection'], key='2e51927d-2798-4afb-9551-d28f6a618c27', value={'content': 'Jim is John Doe's friend.'}, created_at='2025-04-13T14:00:36.853164+00:00', updated_at='2025-04-13T14:00:36.853170+00:00', score=None)]
```

```
In [42]: store.search(('email_assistant', 'lance', 'collection'), query="salman")
```

```
Out[42]: [Item(namespace=['email_assistant', 'lance', 'collection'], key='2e51927d-2798-4afb-9551-d28f6a618c27', value={'content': 'Jim is John Doe's friend.'}, created_at='2025-04-13T14:00:36.853164+00:00', updated_at='2025-04-13T14:00:36.853170+00:00', score=0.14218848233502732)]
```

Create the rest of the agent

```
In [43]: from langgraph.graph import add_messages
```

```
class State(TypedDict):
    email_input: dict
    messages: Annotated[list, add_messages]
```

```
In [44]: from langgraph.graph import StateGraph, START, END
from langgraph.types import Command
from typing import Literal
from IPython.display import Image, display
```

```
In [45]: def triage_router(state: State) -> Command[
    Literal["response_agent", "__end__"]
]:
    author = state['email_input']['author']
    to = state['email_input']['to']
    subject = state['email_input']['subject']
    email_thread = state['email_input']['email_thread']

    system_prompt = triage_system_prompt.format(
        full_name=profile["full_name"],
        name=profile["name"],
        user_profile_background=profile["user_profile_background"],
        triage_no=prompt_instructions["triage_rules"]["ignore"],
        triage_notify=prompt_instructions["triage_rules"]["notify"],
        triage_email=prompt_instructions["triage_rules"]["respond"],
        examples=None
    )
    user_prompt = triage_user_prompt.format(
        author=author,
        to=to,
        subject=subject,
        email_thread=email_thread
    )
    result = llm_router.invoke(
        [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ]
    )
    if result.classification == "respond":
        print("📧 Classification: RESPOND - This email requires a response")
        goto = "response_agent"
        update = {
            "messages": [
                {
                    "role": "user",
                    "content": f"Respond to the email {state['email_input']}"
                }
            ]
        }
    elif result.classification == "ignore":
        print("🚫 Classification: IGNORE - This email can be safely ignored")
        update = None
        goto = END
    elif result.classification == "notify":
        # If real life, this would do something else
```



```

print("🔔 Classification: NOTIFY - This email contains important info")
update = None
goto = END
else:
    raise ValueError(f"Invalid classification: {result.classification}")
return Command(goto=goto, update=update)

```

Create email agent

```

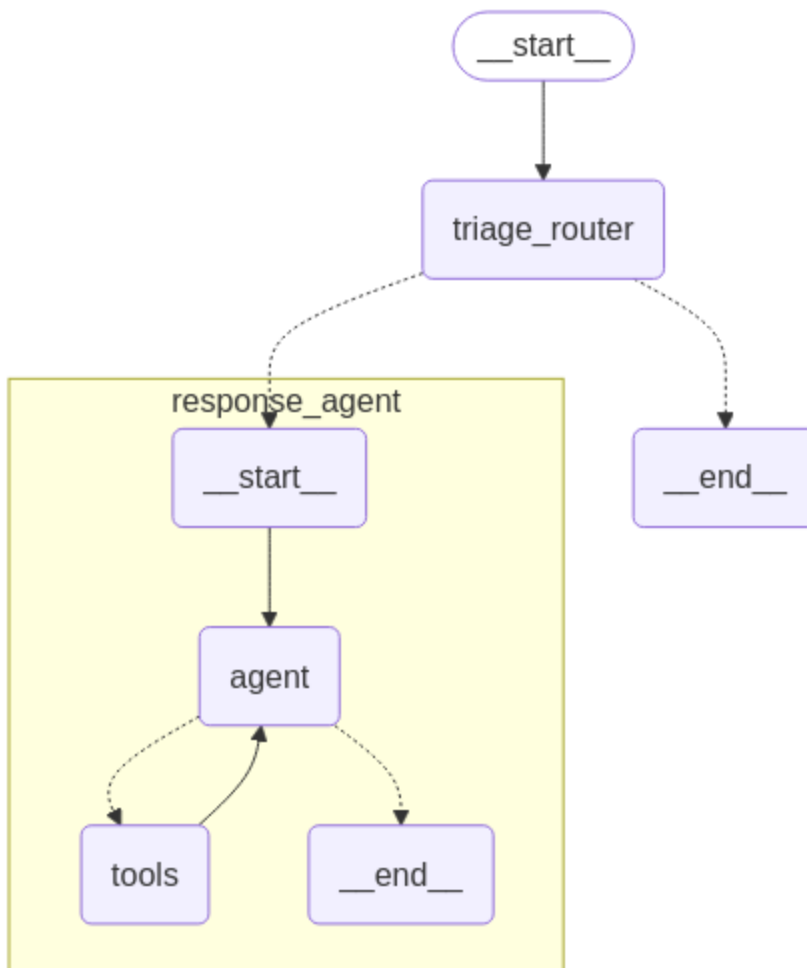
In [46]: email_agent = StateGraph(State)
email_agent = email_agent.add_node(triage_router)
email_agent = email_agent.add_node("response_agent", response_agent)
email_agent = email_agent.add_edge(START, "triage_router")
email_agent = email_agent.compile(store=store)

```

```

In [47]: display(Image(email_agent.get_graph(xray=True).draw_mermaid_png()))

```



```

In [48]: email_input = {
    "author": "Alice Smith <alice.smith@company.com>",
    "to": "John Doe <john.doe@company.com>",
    "subject": "Quick question about API documentation",
    "email_thread": """"Hi John,

```

I was reviewing the API documentation for the new authentication service and

Specifically, I'm looking at:

- /auth/refresh
- /auth/validate

Thanks!

Alice""",
}

```
In [49]: response = email_agent.invoke(  
        {"email_input": email_input},  
        config=config  
        )
```

 Classification: RESPOND – This email requires a response

```
In [50]: for m in response["messages"]:  
        m.pretty_print()
```

===== Human Message =====
=====

Respond to the email {'author': 'Alice Smith <alice.smith@company.com>', 'to': 'John Doe <john.doe@company.com>', 'subject': 'Quick question about API documentation', 'email_thread': "Hi John,\n\nI was reviewing the API documentation for the new authentication service and noticed a few endpoints seem to be missing from the specs. Could you help clarify if this was intentional or if we should update the docs?\n\nSpecifically, I'm looking at:\n- /auth/refresh\n- /auth/validate\n\nThanks!\nAlice"}

===== Ai Message =====
=====

[{'text': "I'll help draft a response to Alice regarding the API documentation question. I'll use the write_email function to send a professional response on John's behalf.", 'type': 'text'}, {'id': 'toolu_01Pc7hT2S1S98ReADHHKcgad', 'input': {'to': 'alice.smith@company.com', 'subject': 'Re: Quick question about API documentation', 'content': "Hi Alice,\n\nI'll need to check with the development team regarding the status of those endpoints in the documentation. Let me look into this and get back to you with a complete answer.\n\nCould we schedule a quick call to discuss this in more detail? That way, we can ensure all necessary endpoints are properly documented.\n\nBest regards,\nJohn"}, 'name': 'write_email', 'type': 'tool_use'}]

Tool Calls:

write_email (toolu_01Pc7hT2S1S98ReADHHKcgad)

Call ID: toolu_01Pc7hT2S1S98ReADHHKcgad

Args:

to: alice.smith@company.com

subject: Re: Quick question about API documentation

content: Hi Alice,

I'll need to check with the development team regarding the status of those endpoints in the documentation. Let me look into this and get back to you with a complete answer.

Could we schedule a quick call to discuss this in more detail? That way, we can ensure all necessary endpoints are properly documented.

Best regards,

John

===== Tool Message =====
=====

Name: write_email

Email sent to alice.smith@company.com with subject 'Re: Quick question about API documentation'

===== Ai Message =====
=====

[{'text': "Let me also schedule a follow-up meeting to discuss this matter more thoroughly. I'll check calendar availability for tomorrow and then schedule a meeting.", 'type': 'text'}, {'id': 'toolu_01EVvZPJUpQdNjAzCci5ZnYF', 'input': {'day': '2024-01-16'}, 'name': 'check_calendar_availability', 'type': 'tool_use'}]

Tool Calls:

check_calendar_availability (toolu_01EVvZPJUpQdNjAzCci5ZnYF)

Call ID: toolu_01EVvZPJUPqdNjAzcCi5ZnYF

Args:

day: 2024-01-16

===== Tool Message =====
=====

Name: check_calendar_availability

Available times on 2024-01-16: 9:00 AM, 2:00 PM, 4:00 PM

===== Ai Message =====
=====

[{'id': 'toolu_01Rt5P5oypp2EMs34xX2pAX1', 'input': {'attendees': ['alice.smith@company.com', 'john.doe@company.com'], 'subject': 'Discuss API Documentation - Authentication Service', 'duration_minutes': 30, 'preferred_day': '2024-01-16'}, 'name': 'schedule_meeting', 'type': 'tool_use'}]

Tool Calls:

schedule_meeting (toolu_01Rt5P5oypp2EMs34xX2pAX1)

Call ID: toolu_01Rt5P5oypp2EMs34xX2pAX1

Args:

attendees: ['alice.smith@company.com', 'john.doe@company.com']

subject: Discuss API Documentation - Authentication Service

duration_minutes: 30

preferred_day: 2024-01-16

===== Tool Message =====
=====

Name: schedule_meeting

Meeting 'Discuss API Documentation - Authentication Service' scheduled for 2024-01-16 with 2 attendees

===== Ai Message =====
=====

[{'text': "I'll also store this interaction in memory for future reference.", 'type': 'text'}, {'id': 'toolu_01SubvzrGTvnF7dKScfYmk3U', 'input': {'action': 'create', 'content': 'Alice Smith raised questions about missing API documentation for authentication service endpoints (/auth/refresh and /auth/validate). Follow-up meeting scheduled for 2024-01-16 to discuss documentation updates.'}, 'name': 'manage_memory', 'type': 'tool_use'}]

Tool Calls:

manage_memory (toolu_01SubvzrGTvnF7dKScfYmk3U)

Call ID: toolu_01SubvzrGTvnF7dKScfYmk3U

Args:

action: create

content: Alice Smith raised questions about missing API documentation for authentication service endpoints (/auth/refresh and /auth/validate). Follow-up meeting scheduled for 2024-01-16 to discuss documentation updates.

===== Tool Message =====
=====

Name: manage_memory

created memory 866ec6ba-a360-44b5-94ff-c4a070df8d56

===== Ai Message =====
=====

I've taken the following actions:

1. Sent a response to Alice acknowledging her question and proposing a meeting

ng

2. Checked calendar availability for tomorrow
3. Scheduled a 30-minute meeting with Alice to discuss the API documentation
4. Created a memory record of this interaction for future reference

The meeting has been scheduled for tomorrow, where John and Alice can discuss the documentation issues in detail and determine the next steps for updating the API documentation.

Try a follow-up email

```
In [54]: email_input = {
    "author": "Alice Smith <alice.smith@company.com>",
    "to": "John Doe <john.doe@company.com>",
    "subject": "Follow up",
    "email_thread": """"Hi John,

Any update on my previous ask?""",
}
```

```
In [ ]: response = email_agent.invoke({"email_input": email_input}, config=config)
```

 Classification: RESPOND – This email requires a response

```
In [ ]: for m in response["messages"]:
    m.pretty_print()
```